

Câu 1: Mini Stationery Api

Một cửa hàng văn phòng phẩm nhỏ cần hệ thống API đơn giản để quản lý:

- + Trả về danh sách các sản phẩm (products) như bút, giấy, dụng cụ vẽ,...
- + Cho biết tồn kho của từng mặt hàng.
- + Phân loại các mặt hàng theo từng nhóm.
- + Lên đơn thanh toán khi khách hàng mua hàng.
- + Cho biết ứng dụng đang chạy ở môi trường nào.
- + Đọc một số cấu hình cơ bản từ appsettings.json.

Models:

Product:

```
namespace AspNetWeek1.Api.Models
{
    18 references
    public class Product
    {
        12 references
        public int Id { get; set; }
        11 references
        public string Name { get; set; } = String.Empty;
        11 references
        public decimal Price { get; set; }
        12 references
        public int StockQuantity { get; set; }
        12 references
        public string CategoryName { get; set; } = String.Empty;
    }
}
```

Category:

```
namespace AspNetWeek1.Api.Models
{
    6 references
    public class Category
    {
        4 references
        public int Id { get; set; }
        5 references
        public string Name { get; set; } = String.Empty;
        1 reference
        public List<Product> Products { get; set; } = new List<Product>();
    }
}
```

Order:

```

namespace AspNetWeek1.Api.Models
{
    4 references
    public class Order
    {
        3 references
        public int Id { get; set; }
        2 references
        public DateTime OrderDate { get; set; }
        2 references
        public decimal TotalAmount { get; set; }
        3 references
        public List<Product> Products { get; set; } = new List<Product>();
    }
}

```

Services:

ProductService:

```

using AspNetWeek1.Api.Models;
namespace AspNetWeek1.Api.Services;

6 references
public class ProductService
{
    2 references
    private readonly CategoryService _categoryService;
    5 references
    private readonly List<Product> _products =
    [
        new Product
        {
            //but chi go faber
            Id = 1,
            Name = "Faber-Castell 9000 Graphite Pencil (HB)",
            Price = 80,
            StockQuantity = 120,
            CategoryName = "Writing Instruments"
        },
        //but da quang zebr
        new Product {
            Id = 2,
            Name = "Zebra Mildliner Highlighter (Set of 5)",
            Price = 180,
            StockQuantity = 40,
            CategoryName = "Writing Instruments"
        },
        //giay sticky note pastel
        new Product {
            Id = 3,
            Name = "Pastel Sticky Notes (3x3)",
            Price = 35,
            StockQuantity = 75,
            CategoryName = "Paper & Notebooks"
        },
        new Product {
            Id = 4,
            Name = "Standard Metal Stapler",
            Price = 50,
            StockQuantity = 40,
            CategoryName = "Desk Accessories"
        },
    ],
}

```

```

        //bo mau nuoc
        new Product{
            Id = 5,
            Name = "Watercolor Paint Set (24 Colors)",
            Price = 120,
            StockQuantity = 30,
            CategoryName = "Art Supplies"
        },
        //So ve phac thao
        new Product{
            Id = 6,
            Name = "Strathmore Sketchbook",
            Price = 22,
            StockQuantity = 10,
            CategoryName = "Art Supplies"
        }
    ];
    // Khoi tao ProductService va gan cac san pham vao cac nhom tuong ung
    0 references
    public ProductService(CategoryService categoryService)
    {
        _categoryService = categoryService;
        AttachCategoryAllProducts();
    }
    //ham gan tung san pham vao nhom tuong ung
    1 reference
    private void AttachCategoryAllProducts()
    {
        foreach (var product in _products)
        {
            _categoryService.SetCategoryByName(product.CategoryName, product);
        }
    }
}

```

```

    //ham lay tat ca cac san pham
    1 reference
    public List<Product> GetAllProducts() => _products;
    //ham lay so luong ton kho cua san pham dua vao ten san pham
    1 reference
    public int GetStockQuantity(string productName)
    {
        var product = _products.FirstOrDefault(p => p.Name == productName);
        if(product == null)
        {
            throw new Exception($"Product with name '{productName}' not found.");
        }
        return product.StockQuantity;
    }
    //ham lay danh muc cua san pham dua vao id san pham
    1 reference
    public string GetCategoryName(int productID)
    {
        var product = _products.FirstOrDefault(p => p.Id == productID);
        if(product == null)
        {
            throw new Exception($"Product with ID '{productID}' not found.");
        }
        return product.CategoryName;
    }
}

```

CategoryService:

```

using AspNetWeek1.Api.Models;

namespace AspNetWeek1.Api.Services;

5 references
public class CategoryService
{
    2 references
    private List<Category> _categories =
    [
        new Category
        {
            Id = 1,
            Name = "Writing Instruments"
        },
        new Category
        {
            Id = 2,
            Name = "Paper & Notebooks"
        },
        new Category
        {
            Id = 3,
            Name = "Desk Accessories"
        },
        new Category
        {
            Id = 4,
            Name = "Art Supplies"
        }
    ];
    1 reference
    public void SetCategoryByName(string name, Product product)
    {
        var category = _categories.FirstOrDefault(c => c.Name == name) ?? throw new Exception($"Category with name '{name}' not found.");
        category.Products.Add(product);
    }
    1 reference
    public List<Category> GetAllCategories() => _categories;
}

```

OrderService:

```

using AspNetWeek1.Api.Models;
namespace AspNetWeek1.Api.Services;

3 references
public class OrderService
{
    3 references
    private readonly List<Order> _orders = [
        new Order
        {
            Id = 1,
            OrderDate = DateTime.Now,
            Products = new List<Product>
            {
                new Product { Id = 1, Name = "Faber-Castell 9000 Graphite Pencil (HB)", Price = 80, StockQuantity = 120, CategoryName = "Writing Instruments" },
                new Product { Id = 3, Name = "Pastel Sticky Notes (3x3)", Price = 35, StockQuantity = 75, CategoryName = "Paper & Notebooks" }
            }
        },
        new Order
        {
            Id = 2,
            OrderDate = DateTime.Now,
            Products = new List<Product>
            {
                new Product { Id = 2, Name = "Zebra Mildliner Highlighter (Set of 5)", Price = 180, StockQuantity = 40, CategoryName = "Writing Instruments" }
            }
        }
    ];
    1 reference
    private void CalculateTotalAmount()
    {
        foreach(var order in _orders)
        {
            order.TotalAmount = order.Products.Sum(p => p.Price);
        }
    }
    1 reference
    public List<Order> GetAllOrders()
    {
        CalculateTotalAmount();
        return _orders;
    }
    1 reference
    public decimal GetTotalAmountByOrderId(int orderId)
    {
        var order = _orders.FirstOrDefault(o => o.Id == orderId) ?? throw new Exception($"Order with ID '{orderId}' not found.");
        return order.TotalAmount;
    }
}

```

Controllers:

ProductController:

```
using AspNetWeek1.Api.Services;
using Microsoft.AspNetCore.Mvc;

namespace AspNetWeek1.Api.Controllers;
[ApiController]
[Route("api/[controller]")]
1 reference
public class ProductController: ControllerBase
{
    4 references
    private readonly ProductService _productService;

    0 references
    public ProductController(ProductService productService)
    {
        _productService = productService;
    }

    // endpoint lay tat ca san pham
    [HttpGet("products")]
    0 references
    public IActionResult GetAllProducts()
    {
        var products = _productService.GetAllProducts()
        .Select(p => new
        {
            p.Id,
            p.Name,
            p.Price,
            p.StockQuantity,
            p.CategoryName
        });
        return Ok(products);
    }
}
```

```
//endpoint lay ten danh muc cua san pham dua vao id san pham
[HttpGet("categoryname/{product_id}")]
0 references
public IActionResult GetCategoryName(int product_id)
{
    try
    {
        var categoryName = _productService.GetCategoryName(product_id);
        return Ok("Category Name: " + categoryName);
    }
    catch(Exception ex)
    {
        return NotFound(ex.Message);
    }
}

// endpoint lay so luong ton kho cua san pham dua vao ten san pham
[HttpGet("stockquantity/{product_name}")]
0 references
public IActionResult GetStockQuantity(string product_name)
{
    try
    {
        var stockQuantity = _productService.GetStockQuantity(product_name);
        return Ok("Stock Quantity: " + stockQuantity);
    }
    catch(Exception ex)
    {
        return NotFound(ex.Message);
    }
}
}
```

CategoryController:

```

using AspNetWeek1.Api.Services;
using Microsoft.AspNetCore.Mvc;

namespace AspNetWeek1.Api.Controllers;
[ApiController]
[Route("api/[controller]")]
1 reference
public class CategoryController: ControllerBase
{
    2 references
    private readonly CategoryService _categoryService;
    1 reference
    private readonly ProductService _productService;

    0 references
    public CategoryController(CategoryService categoryService, ProductService productService)
    {
        _categoryService = categoryService;
        _productService = productService;
    }

    //endpoint lay tat ca danh muc va thong tin tat ca san pham trong tung danh muc
    [HttpGet("categories")]
    0 references
    public IActionResult GetAllCategories()
    {
        var categories = _categoryService.GetAllCategories();
        return Ok(categories);
    }
}

```

OrderController:

```

using AspNetWeek1.Api.Services;
using Microsoft.AspNetCore.Mvc;

namespace AspNetWeek1.Api.Controllers;
[ApiController]
[Route("api/[controller]")]
1 reference
public class OrderController : ControllerBase
{
    3 references
    private readonly OrderService _orderService;

    0 references
    public OrderController(OrderService orderService)
    {
        _orderService = orderService;
    }

    //endpoint lay tat ca don hang
    [HttpGet("orders")]
    0 references
    public IActionResult GetAllOrders()
    {
        var orders = _orderService.GetAllOrders();
        return Ok(orders);
    }

    //endpoint lay tong so tien cua don hang dua vao id don hang
    [HttpGet("totalamount/{orderId}")]
    0 references
    public IActionResult GetTotalAmountByOrderId(int orderId)
    {
        try
        {
            var totalAmount = _orderService.GetTotalAmountByOrderId(orderId);
            return Ok("Total Amount: " + totalAmount);
        }
        catch (Exception ex)
        {
            return NotFound(ex.Message);
        }
    }
}

```

Program.cs:

```

using AspNetWeek1.Api.Models;
using AspNetWeek1.Api.Services;

var builder = WebApplication.CreateBuilder(args);

// Read config and environment
Console.WriteLine($"Environment: {builder.Environment.EnvironmentName}");
Console.WriteLine($"Application: {builder.Environment.ApplicationName}");

// Register services
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();
builder.Services.AddSingleton<ProductService>();
builder.Services.AddSingleton<CategoryService>();
builder.Services.AddSingleton<OrderService>();
builder.Services.Configure<AppSettings>(builder.Configuration.GetSection("AppSettings"));

var app = builder.Build();

//Middleware pipeline
if(app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
app.UseHttpsRedirection();
app.UseStaticFiles();
app.UseAuthorization();

// Minimal API endpoints
app.MapGet("/", () => Results.Ok(new {message = "AspNet Week 1 Mini Stationery Store API is running"}));
// app.MapGet("/")

app.MapGet("/env", () => Results.Ok(new
{
    Environment = app.Environment.EnvironmentName,
}));
app.MapGet("/config", (IConfiguration config) => Results.Ok(new
{
    AppName = config["AppSettings:ApplicationName"],
    Version = config["AppSettings:BaseUr1"]
}));

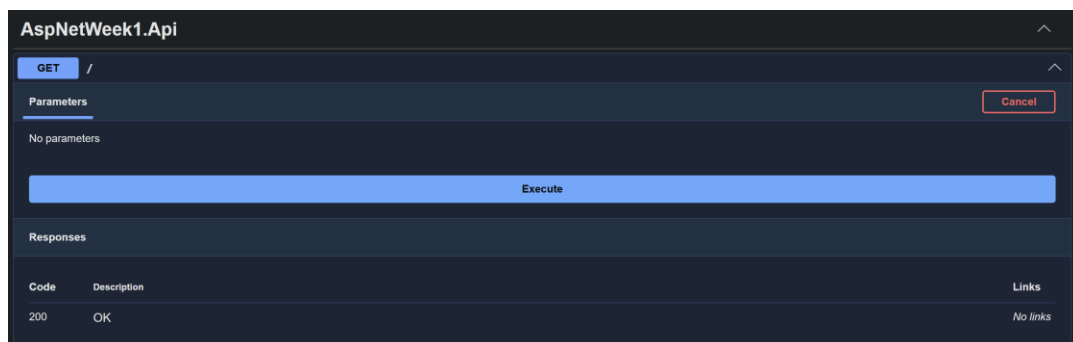
//Controller endpoints
app.MapControllers();

app.Run();

```

Các chức năng:

1. Endpoint Get “/” để kiểm tra app chạy



Sau khi Execute:

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5136/' \
  -H 'accept: */*'
```

Request URL

http://localhost:5136/

Server response

Code	Details
200	Response body <pre>{ "message": "AspNet Week 1 Mini Stationery Store API is running" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Fri, 17 Apr 2026 18:00:07 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK	No links

2. Endpoint Get “/env” để kiểm tra xem là môi trường hiện tại là Production hay Development?

GET /env

Parameters

No parameters

Execute

Responses

Code	Description	Links
200	OK	No links

Sau khi execute:

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5136/env' \
  -H 'accept: */*'
```

Request URL

http://localhost:5136/env

Server response

Code	Details
200	Response body <pre>{ "environment": "Development" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Fri, 17 Apr 2026 18:25:50 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description	Links
200	OK	No links

3. Endpoint Get “/config” dùng để kiểm tra appName và version

GET /config

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5136/config' \
  -H 'accept: */*'
```

Request URL

http://localhost:5136/config

Server response

Code	Details
200	Response body <pre>{ "appName": "ASP.NET Week 1 Mini Stationery Store API Development", "version": "https://localhost:5001" }</pre> Response headers <pre>content-type: application/json; charset=utf-8 date: Fri, 17 Apr 2026 17:08:08 GMT server: Kestrel transfer-encoding: chunked</pre>

Responses

Code	Description	Links
------	-------------	-------

4. Các endpoints của Product:

+Lấy danh sách các products:

GET /api/Product/products

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5136/api/Product/products' \
  -H 'accept: */*'
```

Request URL

http://localhost:5136/api/Product/products

Server response

Code	Details
200	Response body <pre>{ "id": 1, "name": "Faber-Castell 9000 Graphite Pencil (HB)", "price": 99, "stockQuantity": 120, "categoryName": "Writing Instruments" }, { "id": 2, "name": "Zebra Mildliner Highlighter (Set of 5)", "price": 280, "stockQuantity": 40, "categoryName": "Writing Instruments" }, { "id": 3, "name": "Pastel Sticky Notes (3x3)", "price": 30, "stockQuantity": 75, "categoryName": "Paper & Notebooks" }</pre>

+Lấy danh mục của product dựa vào product_id

GET /api/Product/categoryname/{product_id}

Parameters

Name	Description
product_id * required integer(int32) (path)	3

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5136/api/Product/categoryname/3' \
  -H 'accept: */*'
```

Request URL

http://localhost:5136/api/Product/categoryname/3

Server response

Code	Details
200	Response body <pre>Category Name: Paper & Notebooks</pre>

+Truy xuất tồn kho của Product dựa vào tên của Product

GET

/api/Product/stockquantity/{product_name}

Parameters

Cancel

Name	Description
product_name * required	Zebra Mildliner Highlighter (Set of 5)
string (path)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5136/api/Product/stockquantity/Zebra%20Mildliner%20Highlighter%20Set%20of%205' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:5136/api/Product/stockquantity/Zebra%20Mildliner%20Highlighter%20Set%20of%205
```

Server response

Code	Details
200	Response body Stock Quantity: 40 Download

5. Endpoint của Category:

Lấy danh sách của category

GET

/api/Category/categories

Responses

Curl

```
curl -X 'GET' \
  'http://localhost:5136/api/Category/categories' \
  -H 'accept: */*'
```

Request URL

```
http://localhost:5136/api/Category/categories
```

Server response

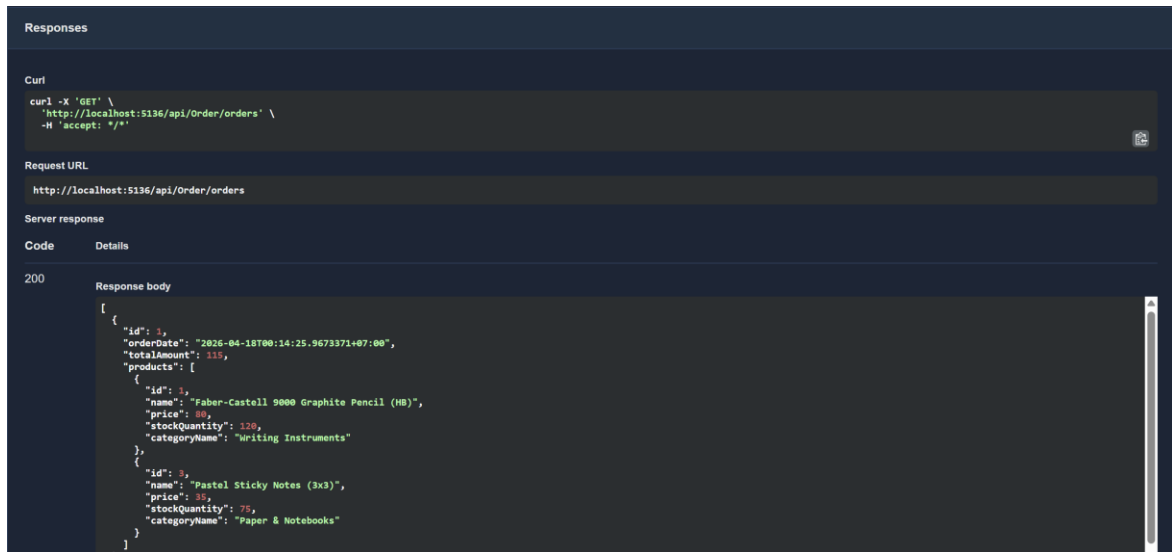
Code	Details
200	Response body <pre>[{ "id": 1, "name": "Writing Instruments", "products": [{ "id": 1, "name": "Faber-Castell 9000 Graphite Pencil (HB)", "price": 80, "stockQuantity": 120, "categoryName": "Writing Instruments" }, { "id": 2, "name": "Zebra Mildliner Highlighter (Set of 5)", "price": 180, "stockQuantity": 40, "categoryName": "Writing Instruments" }] }]</pre>

6. Các Endpoints của Order:

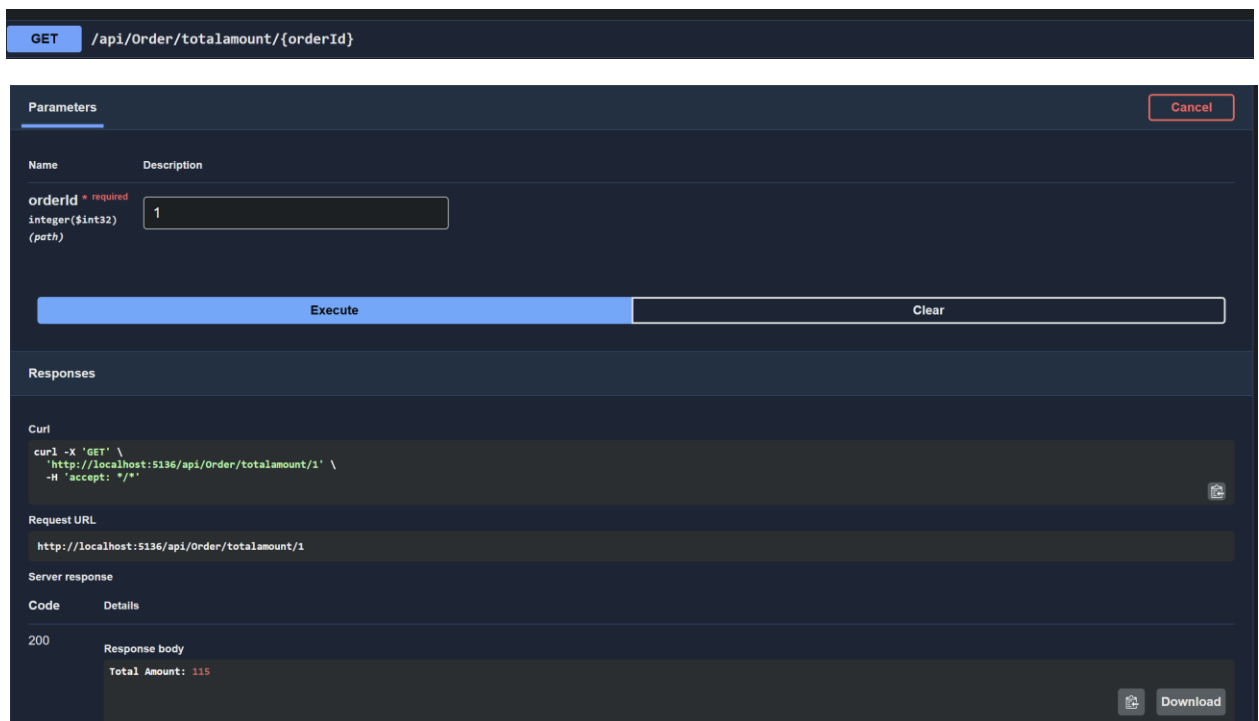
+ Lấy danh sách các order:

GET

/api/Order/orders



+ Truy xuất tổng tiền của đơn hàng dựa vào orderId



Xem chi tiết source code tại: https://github.com/tin1508/Asp.Net_TTH.git

Câu 2:

1. Nếu dữ liệu ngày càng nhiều, việc lưu bằng List hoặc dữ liệu hard-code trong service sẽ gặp những hạn chế gì?

Nếu dữ liệu ngày càng nhiều (1000 loại giấy khác nhau, 100 mẫu bút khác nhau,...), việc lưu bằng List hoặc dữ liệu hard-code trong service sẽ gặp những hạn chế sau:

+ Tốn bộ nhớ Ram, vì dữ liệu hardcode hay List sẽ được lưu trên Ram, vì vậy nên khi quá nhiều dữ liệu thì sẽ tốn rất nhiều bộ nhớ Ram, điều này làm ảnh hưởng đến hiệu năng của hệ thống và trải nghiệm của người dùng khi dùng chương trình.

+ Dữ liệu sẽ không được lưu trữ khi khởi động chương trình lại, ví dụ cập nhật giá tiền cho cây bút mực thì khi khởi động lại chương trình thì giá tiền vẫn là giá tiền của dữ liệu hardcode chứ không phải dữ liệu mới cập nhật.

2. Nếu có nhiều người cùng sử dụng hệ thống, chương trình hiện tại còn thiếu những chức năng nào để dùng được trong thực tế?

Để có nhiều người cùng sử dụng hệ thống, chương trình hiện tại cần:

- + Thêm các thao tác Create, Update, Delete vì hiện tại chỉ có phương thức Get.
- + Xác thực và phân quyền người dùng: hiện tại ai cũng có thể truy cập các endpoints, nhưng thực tế chỉ nhân viên bán hàng hoặc chủ cửa hàng thì mới được sửa hay cập nhật mặt hàng hoặc tồn kho. Khách hàng chỉ được xem thôi. Và điều này cũng hạn chế lượng truy cập cùng lúc vào cùng 1 endpoint quá tải nữa.

3. Nếu muốn giảm sai sót khi nhập dữ liệu hoặc trả sai trạng thái, em sẽ cải tiến phần validation, service logic, routing, middleware hoặc configuration như thế nào?

+ **Validation:** Sử dụng **Data Annotations** để ngăn chặn các lỗi nhập liệu như tên trống hoặc giá tiền hay số lượng tồn kho âm.

+ **Service Logic:** kiểm tra nghiệp vụ tầng Service. Ví dụ: Trước khi trừ kho, Service phải kiểm tra xem số lượng tồn kho có đủ cung cấp cho đơn hàng hay không.

Middleware: Cài đặt một **Global Exception Handler** để bắt lỗi tập trung, vì nếu như dự án có hàng chục hay hàng trăm file, và có những exception giống nhau được sử dụng ở nhiều file thì nếu muốn thay đổi hay chỉnh sửa exception thì phải chỉnh hết 1 hoặc 100 hay 1000 exceptions, nên **Global Exception Handler** sẽ giúp giải quyết được việc này.

4. Nếu cần phát triển ứng dụng này thành một hệ thống thật ở các tuần sau, theo em nên ưu tiên mở rộng phần nào trước: CRUD, tìm kiếm, phân trang, authentication/authorization, hay kết nối database? Hãy giải thích lý do.

Em ưu tiên **kết nối Database** vì những mặt hạn chế ở câu 1. Nếu không có Database, thì việc CRUD, tìm kiếm, phân trang... cũng vô nghĩa, vì dữ liệu đều mất sạch khi chạy lại chương trình.

5. Theo em, trong bài toán em chọn, đâu là vấn đề nghiệp vụ quan trọng nhất mà hệ thống cần giải quyết tốt để có ích trong thực tế?

Theo em, vấn đề nghiệp vụ quan trọng nhất mà hệ thống cửa hàng văn phòng phẩm cần quản lý là quản lý tồn kho và cảnh báo số lượng tồn kho thấp. Mục đích để ngăn chặn nếu người dùng đang cần mua hàng nhưng lại không có hàng. Vì điều này sẽ gây mất lòng khách hàng và dẫn đến sự suy giảm khách hàng đối với cửa hàng, cũng làm giảm luôn doanh thu.

6. Nếu ứng dụng chạy đúng ở local nhưng lên máy khác hoặc môi trường khác thì lỗi, em sẽ kiểm tra theo thứ tự nào giữa:

environment
appsettings.json
appsettings.Development.json
launch profile
middleware pipeline
route mapping

Đầu tiên, em sẽ kiểm tra launch.json vì file này là nơi cấu hình cổng port hay giao thức (Http/Https). Ví dụ: kiểm tra cổng (Port) có bị trùng với ứng dụng khác trên máy mới không. Nếu trùng thì sẽ gây ra lỗi khiến ứng dụng không thể khởi động.

Thứ 2, tiếp theo kiểm tra environment bằng cách xác nhận biến môi trường ASPNETCORE_ENVIRONMENT đã được đặt là Development chưa. Vì nếu không phải development thì **Swagger** sẽ bị tắt theo mặc định trong Program.cs. Nếu không bật đúng môi trường, ta sẽ thấy trang trắng hoặc lỗi.

Thứ 3, kiểm tra file appsettings.Development.json có bị thiếu hay sai ở đâu không? Nếu thiếu file này, các cấu hình riêng cho môi trường Dev sẽ bị mất, khiến các Service không

Thứ 4, khởi tạo được dữ liệu mẫu cho cửa hàng văn phòng phẩm.

kiểm tra middleware pipeline, kiểm tra xem các cấu hình bảo mật (CORS, HTTPS) có đang chặn request từ môi trường mới không. Vì có thể app chạy nhưng báo lỗi 403 (Not found), 404 (bad request),...

Thứ 5, kiểm tra route mapping vì nếu chạy trên môi trường môi trường production thì route của nó có khi sẽ thay đổi. Ví dụ ở local là api/v1/products còn productions là /products.

Cuối cùng, kiểm tra file appsettings.json. Kiểm tra thông số cấu hình chung của hệ thống. Ví dụ endpoint /config cho chúng ta thông tin sai về applicationName hay version...